



FİNAL PROJE RAPORU

Dönem: Bahar 2026

Ders: EFC304 Mobil Uygulama Tasarımı ve Geliştirme

Öğretim Elemanı: Buket İŞLER

Proje Başlığı: Fluxora

Proje konusu: Üniversite Topluluk Ajandası

Öğrenci Adı: Şevval

Soyadı: ARSLAN

Numarası: 22040301030

Bölümü: Yazılım Mühendisliği

E-posta: sevvalarslan@stu.topkapi.edu.tr

GitHub Repo Bağlantısı:

<https://github.com/iamsevval/Fluxora>

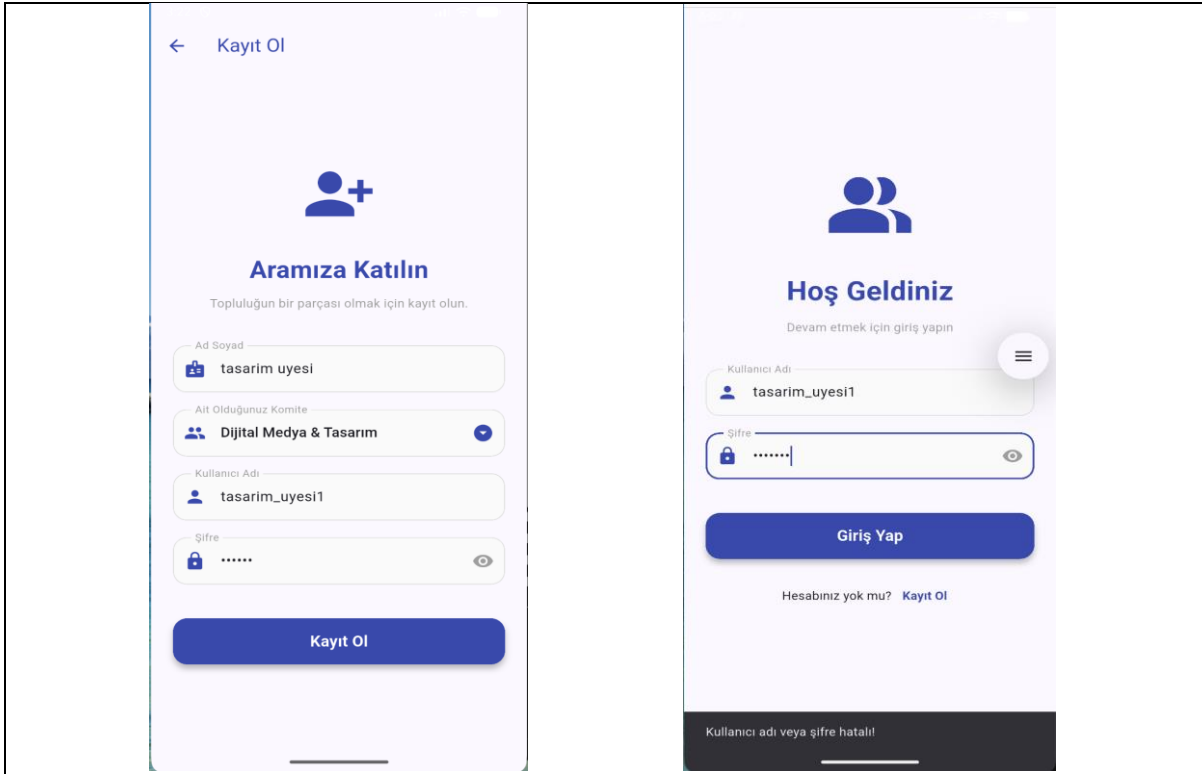
1) GİRİŞ VE PROJE ÖZETİ

Fluxora (Üniversite Topluluk Ajandası), öğrenci topluluklarının iç koordinasyonunu, komite süreçlerini ve zirve organizasyonlarını tek bir merkezden yöneten yerel (native) bir mobil koordinasyon uygulamasıdır. Proje, Flutter SDK ve Dart programlama diliyle geliştirilmiş olup; verilerin güvenli, ilişkisel ve tamamen çevrimdışı (offline-first) saklanması amacıyla SQLite (sqflite) yerel veritabanı mimarisi üzerine inşa edilmiştir. Modüler yazılım mimarisinde (Separation of Concerns) veritabanı erişimi Singleton Tasarım Deseni (DbHelper) ile optimize edilerek normalize edilmiş tablolarla veri bütünlüğü sağlanmıştır.

Uygulamanın kullanıcı arayüzü (UI), modern Material Design ilkeleri, kalıcı Dark/Light Mode desteği ve komitelere özel renk paletleriyle native olarak tasarlanmıştır. Temel SQLite CRUD işlemlerinin yanı sıra; Kampüs Elçisi grafiksel genel bakış paneli, dinamik üye atamalı görev havuzu ve girdi doğrulamalı (validation) Zirve katılım takip modülü gibi ileri seviye teknikler entegre edilmiştir.

2) KULLANICI GİRİŞ (LOGIN) VE KAYIT (REGISTER) EKRANI

Giriş ve kayıt ekranları, topluluk üyelerinin rollerine ve yetkilerine göre ilgili panellere yönlendirilmesini sağlar. Kayıt esnasında üyenin ait olduğu komite seçilir ve bu bilgi SQLite users tablosuna yazılır. Giriş esnasında girilen kullanıcı adı ve şifre, SQLite tablosunda sorgulanır (loginUser). Eşleşme başarılı ise kullanıcı nesnesi (User) oluşturularak anasayfaya rol ve renk parametreleriyle birlikte aktarılır. Hatalı girişlerde kullanıcı SnackBar ile uyarılır.



```
// lib/database/db_helper.dart - SQLite Kullanıcı Giriş Sorgusu
Future<User?> loginUser(String username, String password) async {
  var dbClient = await db;
  List<Map<String, dynamic>> result = await dbClient.query(
    'users',
    where: 'username = ? AND password = ?',
    whereArgs: [username, password], // SQL Injection engelleme
  );

  if (result.isNotEmpty) {
    return User.fromMap(result.first);
  }
  return null;
}

// lib/screens/login_screen.dart - Giriş Butonu Tetikleyicisi
void _login() async {
  if (_formKey.currentState!.validate()) {
    setState(() => _isLoading = true);
  }
}
```

```

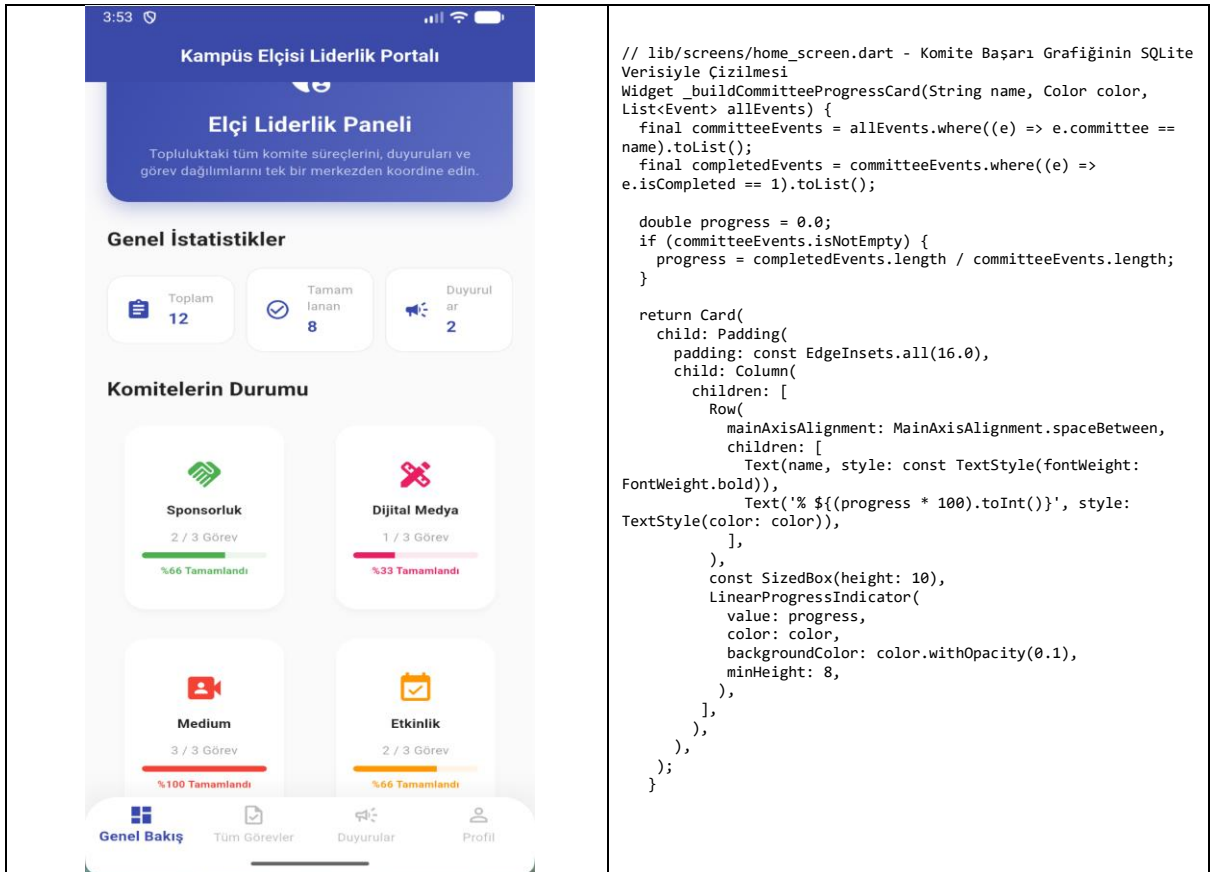
User? user = await _dbHelper.loginUser(
  _usernameController.text.trim(),
  _passwordController.text,
);
setState(() => _isLoading = false);

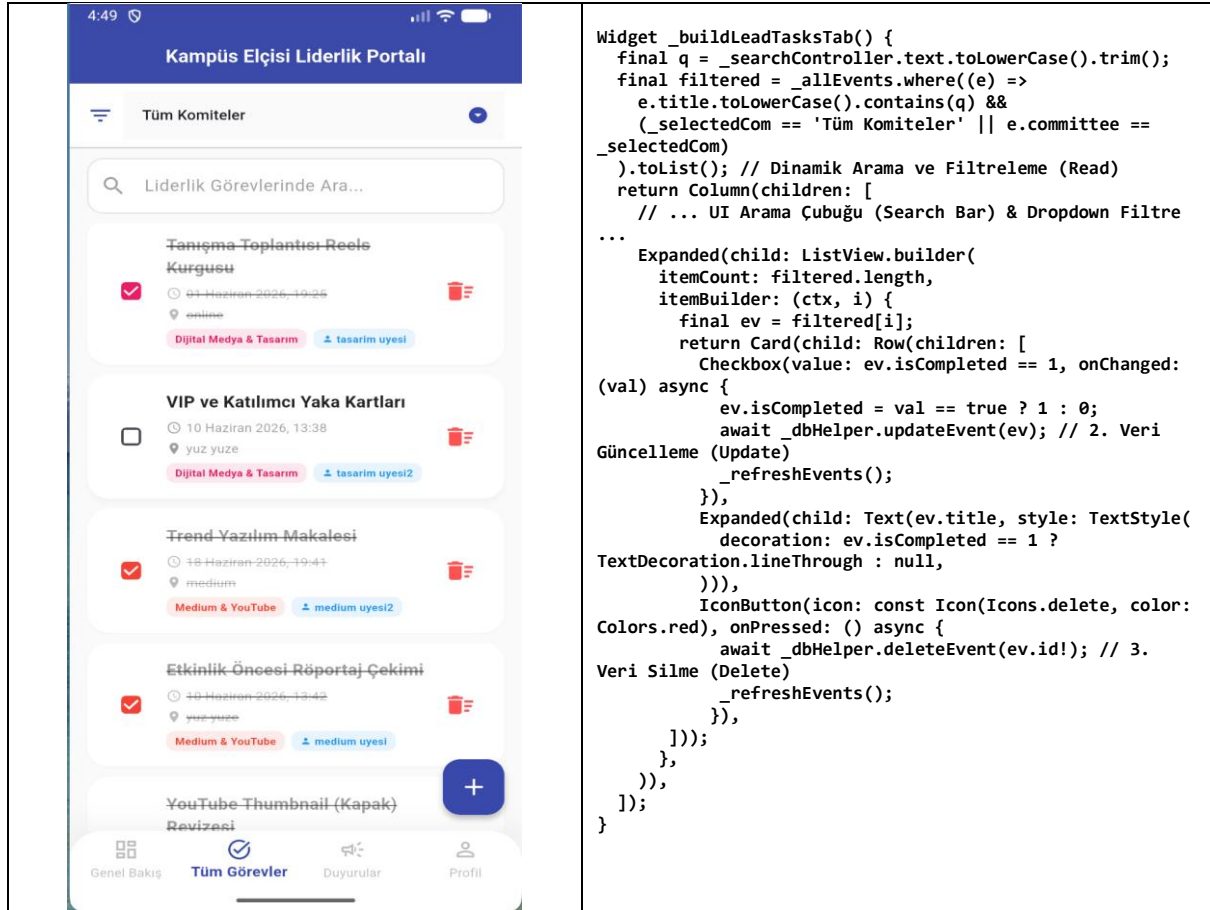
if (user != null) {
  // Giriş başarılı, ana ekrana yönlendir
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(
      builder: (context) => HomeScreen(user: user, committeeName: user.primaryCommittee!),
    ),
  );
} else {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Kullanıcı adı veya şifre hatalı!')),
  );
}
}
}

```

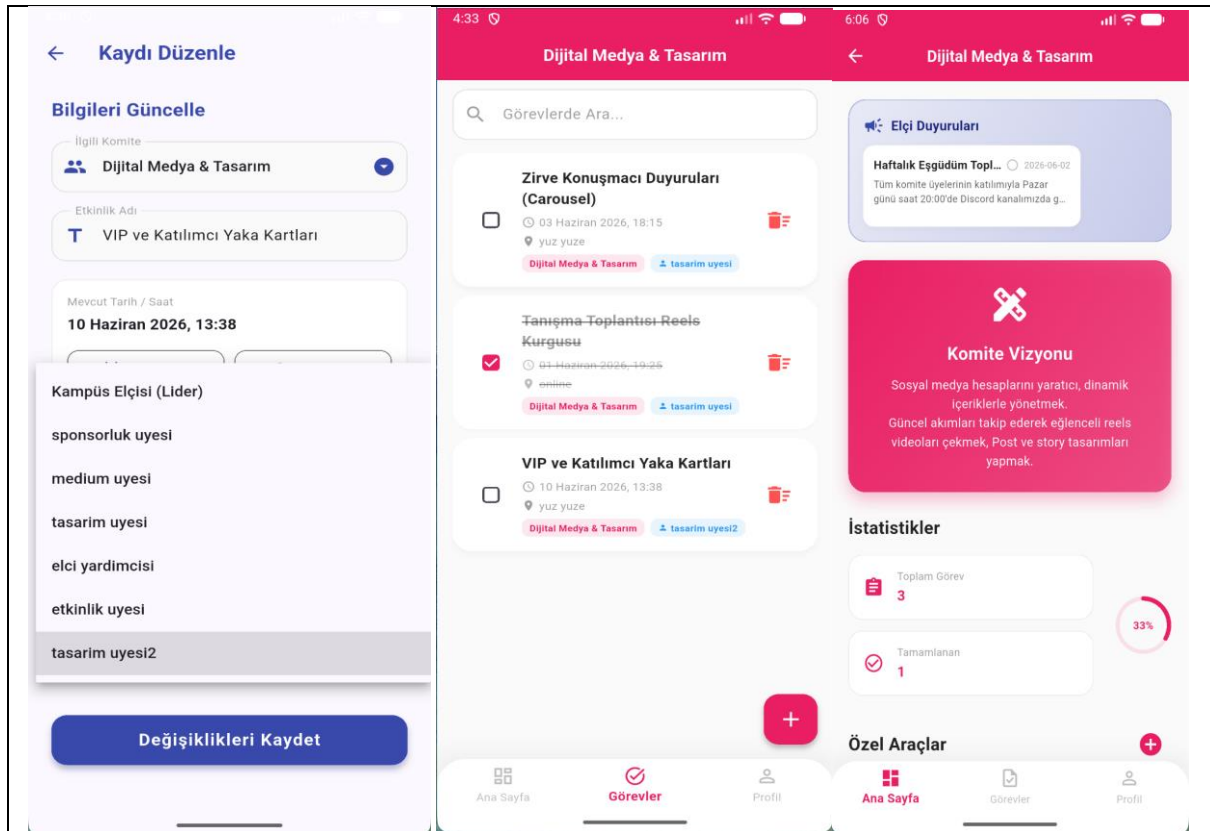
3) KAMPÜS ELÇİSİ (LİDER) PORTALI VE MERKEZİ TAKİP

Liderlik Portalı, topluluk yöneticilerinin (Kampüs Elçisi) tüm komitelerin durumunu gerçek zamanlı izlemesini sağlar. SQLite üzerindeki events tablosunda yer alan görev sayıları komite adına göre gruplanarak okunur. Toplam görev ve tamamlanan görev oranlarına göre LinearProgressIndicator bileşeni kullanılarak dinamik bir başarı grafiği çizdirilir. Grafik kartlarına tıklandığında, sistem görevleri otomatik olarak o komiteye göre filtreler.





4) KİŞİYE ÖZEL GÖREV ATAMA VE DUYURU SİSTEMİ



```
// lib/screens/add_event_screen.dart - Dinamik Üye Listesini SQLite'tan Çekme ve Dropdown
List<User> _users = [];
String? _selectedUser;

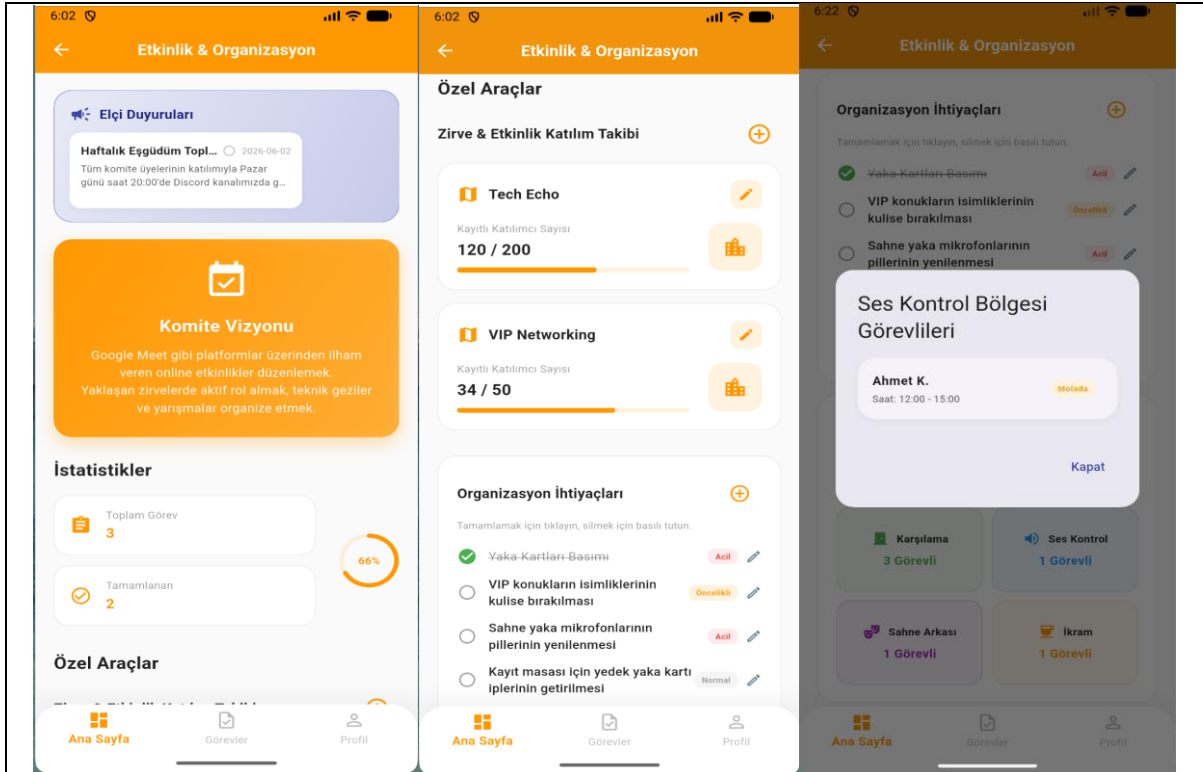
void _loadUsers() async {
  final users = await _dbHelper.getAllUsers(); // SQLite'tan tüm kullanıcıları çek
  setState(() {
    _users = users;
  });
}

// Dropdown Arayüz Tanımı
DropdownButtonFormField<String>({
  value: _selectedUser,
  decoration: const InputDecoration(labelText: 'Görevi Atayacağın Kişi (Opsiyonel)'),
  items: _users.map((User u) {
    return DropdownMenuItem<String>({
      value: u.fullName,
      child: Text(u.fullName),
    });
  }).toList(),
  onChanged: (String? newValue) {
    setState(() => _selectedUser = newValue);
  },
});
```

Görevlerin kime atandığını statik metinler yerine SQLite veritabanındaki gerçek kayıtlarla eşleştirmek amacıyla dinamik üye listesi entegrasyonu yapılmıştır. `add_event_screen.dart` açıldığında `DbHelper.getAllUsers()` metodu çalışarak tüm kayıtlı üyeleri çeker ve görev atama dropdown listesine bağlar. Görev oluşturulduğunda üyenin adı `assignedTo` hüccesine kaydedilir. Duyuru modülünde ise kampüs elçisinin yayınladığı genel veya komiteye özel duyurular SQLite'tan dinamik olarak okunarak ilgili üyenin panosunda kayan bant halinde gösterilir.

5) ZİRVE KATILIM TAKİBİ VE VERİ DOĞRULAMA (CRUD)

Zirve ve etkinlik organizasyonlarının sürdürülebilirliği için doluluk takip ekranı tasarlanmıştır. SQLite üzerinde `committee_items` tablosunda `summit_capacity` tipiyle saklanan veriler, arayüze çekilerek oranlanır. Veritabanına kayıt yazılmadan önce iş mantığı katmanında doğrulama (Validation) filtreleri uygulanır: Katılımcı sayısının negatif olması, maksimum kapasiteden büyük girilmesi veya kapasitenin sıfır ya da altında olması durumunda SQLite işlemi iptal edilir ve kullanıcıya açıklayıcı hata `SnackBar`'ları fırlatılır.



```
// 1. DİNAMİK EKİP GÖREV MATRİSİ (DASHBOARD)
Widget _buildDutyMatrix() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text('Ekip Görev Matrisi', style: TextStyle(fontWeight: FontWeight.bold, fontSize: 16)),
    ],
  );
}
```

```

const Text('Zirve günü alanlarındaki görev dağılımlarını anlık izleyin.'),

// Responsive Izgara (Grid) Yapısı
GridView.count(
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  crossAxisCount: 2, // Yan yana 2 kutu
  childAspectRatio: 1.6,
  children: [
    _buildZoneCard('Karşılama', Icons.door_front_door, Colors.green),
    _buildZoneCard('Ses Kontrol', Icons.volume_up, Colors.blue),
    _buildZoneCard('Sahne Arkası', Icons.theater_comedy, Colors.purple),
    _buildZoneCard('İkram', Icons.coffee, Colors.orange),

    ], ), ], ); }

// 2. HER BİR BÖLGE İÇİN KART OLUŞTURUCU METOT
Widget _buildZoneCard(String zoneName, IconData icon, Color color) {
  // Veritabanından (veya state'ten) gelen listede bu bölgeye ait kaç kişi olduğunu hesapla
  int staffCount = _duties.where((duty) => duty.dutyZone == zoneName).length;

  return InkWell(
    onTap: () => _showZoneStaffDetails(zoneName), // Tıklayınca detay listesini aç
    child: Container(
      decoration: BoxDecoration(
        color: color.withOpacity(0.06),
        borderRadius: BorderRadius.circular(15),
        border: Border.all(color: color.withOpacity(0.2)),
      ),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(icon, color: color, size: 20),
          Text(zoneName, style: const TextStyle(fontWeight: FontWeight.bold)),
          const SizedBox(height: 6),
          // Canlı İstatistik Gösterimi
          Text(
            '$staffCount Görevli',
            style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold, color: color),
          ),
        ], ), ), ); }

// lib/database/db_helper.dart - SQLite Temel CRUD (Ekle, Oku, Güncelle, Sil) İşlemleri
Future<int> insertEventDuty(EventDuty duty) async {
  var dbClient = await db;
  return await dbClient.insert('event_duties', duty.toMap()); // CREATE
}

Future<List<EventDuty>> getEventDuties() async {
  var dbClient = await db;
  List<Map<String, dynamic>> maps = await dbClient.query('event_duties'); // READ
  return List.generate(maps.length, (i) => EventDuty.fromMap(maps[i]));
}

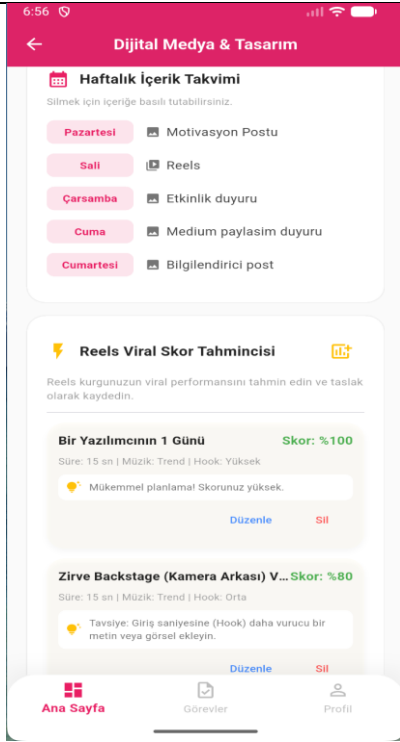
Future<int> updateEventDuty(EventDuty duty) async {
  var dbClient = await db;
  return await dbClient.update('event_duties', duty.toMap(),
    where: 'id = ?', whereArgs: [duty.id]); // UPDATE
}

Future<int> deleteEventDuty(int id) async {
  var dbClient = await db;
  return await dbClient.delete('event_duties',
    where: 'id = ?', whereArgs: [id]); // DELETE
}

```

6) GELİŞMİŞ KOMİTE ÖZEL ARAÇLARI VE ALGORİTMALAR

Uygulama, komitelere değer sağlamak amacıyla matematiksel algoritmalara dayalı iki ana modül içerir: Reels Viral Skor Motoru, video parametrelerine (süre, trend müzik, kanca gücü) göre anlık viralite hesaplaması yaparak veritabanına kaydeder. Sponsorluk Paket Hesaplayıcısı ise taban bütçe ve seçilen ek donanımlara (stant, logo, post) göre dinamik fiyatlandırma çarpanı oluşturarak akıllı paket tasarımları sunar.

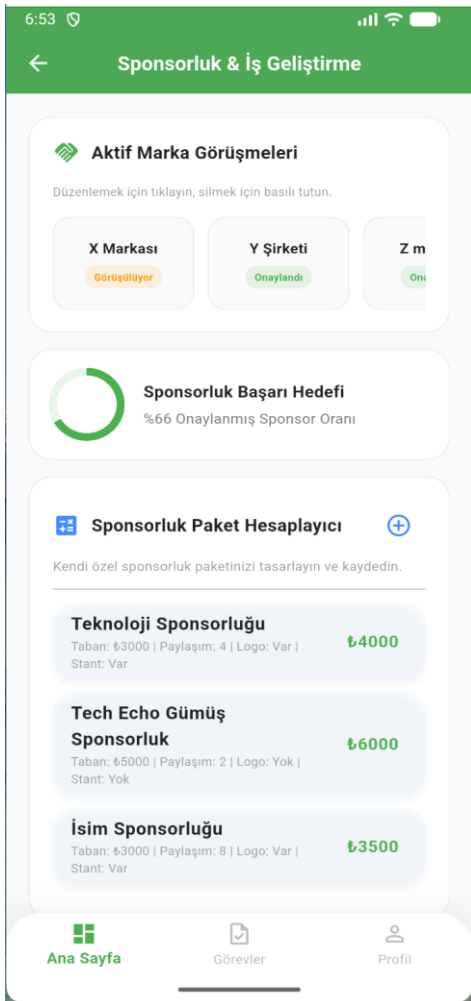


```
// lib/screens/home_screen.dart - Reels Viral Skor ve Tavsiye Algoritması
int score = 0;
// Süre Analizi (Kısa videolar daha yüksek puan alır)
if (duration <= 15) score += 40;
else if (duration <= 30) score += 20;
else score += 5;

// Trend Müzik Çarpanı
if (isTrendingMusic) score += 20;

// Etkileşim Skoru
if (hookStrength == 'Yüksek') score += 40;
else if (hookStrength == 'Orta') score += 20;
else score += 5;

// DİNAMİK TAVSİYE MOTORU
String recs = 'Mükemmel planlama! Skorunuz yüksek.';
if (duration > 15) {
  recs = 'Tavsiye: Süreyi 15 saniyenin altına çekerek izlenme
  tamamlama oranını artırın.';
} else if (!isTrendingMusic) {
  recs = 'Tavsiye: Keşfet etkileşimi yakalamak için mutlaka güncel
  bir Trend ses ekleyin.';
} else if (hookStrength != 'Yüksek') {
  recs = 'Tavsiye: Giriş saniyesine (Hook) daha vurucu bir metin
  veya görsel ekleyin.';
}
```



```
// lib/screens/home_screen.dart - Dinamik Sponsorluk Fiyat Hesaplayıcı
ve State Yönetimi
showDialog(
  context: context,
  builder: (context) => StatefulBuilder(
    builder: (context, setState) => {

      // Dinamik Fiyat Çarpanı (Ekrandaki State değiştiğinde anında
      güncellenir)
      double totalPrice = budgetLimit
        + (socialMediaPosts * 500)
        + (logoBanner ? 1500 : 0)
        + (standArea ? 3000 : 0);

      return AlertDialog(
        title: const Text('Sponsorluk Paket Tasarlayıcı'),
        content: Column(
          children: [
            // --- FLUTTER UI: SLIDER (Taban Bütçe Belirleme) ---
            Slider(
              value: budgetLimit,
              min: 1000, max: 20000, divisions: 19,
              onChanged: (val) {
                setState(() => budgetLimit = val); // Kullanıcı
                kayırdıkça UI tetiklenir
              },
            ),
            // FLUTTER UI: CHECKBOX LIST TILE
            CheckboxListTile(
              title: const Text('Afis/Banner Logo Görseli'),
              value: logoBanner,
              onChanged: (val) => setState(() => logoBanner =
                val!),
            ),
            CheckboxListTile(
              title: const Text('Zirve Alanı Stant Kurulumu'),
              value: standArea,
              onChanged: (val) => setState(() => standArea =
                val!),
            ),
            // Algoritmanın anlık olarak Text (Metin) Widget'ına
            yansıtılması
            Text('Hesaplanan Değer: ₺${totalPrice.toInt()}',
              style: const TextStyle(fontWeight: FontWeight.bold,
                fontSize: 18),
            ),
          ],
        ),
      );
    },
  ),
);
```

7) HATA KONTROLLERİ (ERROR HANDLING) VE VERİ DOĞRULAMA

Uygulama genelinde kullanıcı deneyimini korumak ve SQLite veritabanı işlemlerinde uygulamanın çökmesini (crash) engellemek amacıyla try-catch blokları kullanılmıştır. Özellikle yeni etkinlik veya duyuru eklenirken boş form gönderilmesini engellemek için Form doğrulama (Validation) işlemleri yapılmış, olası hatalar ekrana ScaffoldMessenger (SnackBar) ile yansıtılmıştır.

	<pre>// lib/screens/add_event_screen.dart - Form Validasyon ve Hata Kontrolü void _saveEvent() async { final title = _titleController.text.trim(); // 1. Boş Alan Kontrolü (Form Validation) if (title.isEmpty _selectedDate == null _selectedTime == null) { ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Lütfen etkinlik adını, tarihini ve saatini seçin.')),); return; // İşlemi durdur, veritabanına yazma } Event newEvent = Event(title: title, date: finalDateTimeString, location: _locationController.text.trim(), description: _descriptionController.text.trim(), committee: _currentCommittee!, assignedTo: _selectedUser,); // 2. Veritabanı Yazma İşlemi ve Try-Catch Bloğu try { await _dbHelper.insertEvent(newEvent); if (mounted) Navigator.pop(context); // Başarılıysa ekranı kapat } catch (e) { if (mounted) { // Olası bir SQLite çökmesinde uygulamayı koru ve hata fırlat ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Kayıt sırasında bir hata oluştu: \$e')),); } } }</pre>
--	--

8) PROJE KAZANIMLARI VE DEĞERLENDİRME

Bu final projesinde, üniversite öğrenci topluluklarının koordinasyon problemlerini çözmek amacıyla SQLite tabanlı yerel veri depolama katmanına sahip native bir mobil uygulama geliştirilmiştir. Proje süresince elde edilen en büyük kazanım, mobil cihazların donanımsal kaynaklarını (bellek, işlemci) optimize ederek internet bağlantısının bulunmadığı durumlarda dahi çalışabilen (Offline-First) bir sistem mimarisi tasarlamak olmuştur.

SQLite Mimarisi ve Veri Kalıcılığı Değerlendirmesi: SQLite yerel veritabanı kullanımı, bulut tabanlı sistemlere kıyasla veri erişim hızını (Latency) neredeyse sıfıra indirmiştir. Veritabanının cihaz yerelinde (topluluk_v14.db) şifreli veya doğrudan barındırılması, ağ dalgalanmalarından kaynaklanan senkronizasyon hatalarını ve veri kayıplarını tamamen engellemiştir. Model-View mimarisi sayesinde verilerin nesne yönelimli programlama (OOP) mantığıyla Map yapılarına dönüştürülüp SQL sorgularıyla işlenmesi, veritabanı normalize etmenin ve ilişkisel tabloların önemini pratik olarak kavramanı sağlamıştır.

Karşılaşılan Zorluklar ve Çözümleri: Geliştirme sürecinde karşılaşılan temel zorluk, veritabanı şema güncellemelerinde (Migration) önceki verilerin silinmeden yeni tablolara aktarılması olmuştur. Bu problem, db_helper.dart içindeki onUpgrade metodu ve veritabanı sürüm kontrol mekanizması (Version Control) kullanılarak çözülmüştür. Sonuç olarak, SQLite, Dart ve Flutter teknolojilerini harmanlayarak, akademik kriterlere bağlı ve sektörel standartlarda bir native mobil uygulama başarıyla tamamlanmıştır.